information. This reduces the risk of hallucinations, where the language model produces plausible-sounding but factually incorrect output.

For the grounding component in our RAG system, we will use the Google Gemini log probabilities support, which allows us to assess the confidence and uncertainty of the system's responses. In the next sections, we'll implement a step-by-step RAG web application using LangChain.js, demonstrating how to create a robust, context-aware generative system.

NOTE The source code for the examples in this chapter is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch07/rag-web-app.

Project code: rag-web-app

The code for this example project is in the ch07/rag-web-app folder. You can use that example by running this command in the root folder of the project repository:

```
$ npm run dev -w ch07/rag-web-app
```

For details on setting up and creating API keys, refer to the appendix:

```
// .env
OPENAI_API_KEY=<OPEN_AI_SECRET_KEY>
GEMINI_API_KEY=<GOOGLE_AI_SECRET_KEY>
```

7.2.4 RAG system components

Since there is quite a bit of code involved when explaining the key components of a RAG system, we will look at the important bits of each component while focusing on the bigger picture. Let's start with the document-indexing process.

Note for running the example script

To test the RAG implementation described in this chapter, first navigate to the script's directory and then run it:

```
$ cd ch07/summarization-examples
$ node rag.js
```

This script provides a practical example of the RAG workflow we've just discussed, allowing you to experiment with the code and see the RAG system in action.

DOCUMENT INDEXING

Document indexing transforms raw text documents into a machine-readable, semantically searchable format. This important first step in the RAG workflow involves converting unstructured documents into high-dimensional vectors.